

PROTOCOLO HTTP

Funcionamento, Evolução e Segurança na Web

Administração de Redes

Emilia Ribeiro Santos

O que é o HTTP?



HyperText Transfer Protocol

O protocolo base para a comunicação na web, definindo como as mensagens são formatadas e transmitidas.



Cliente-Servidor

Gerencia a troca de dados entre navegadores (clientes) e servidores web através de um modelo de requisição e resposta.



Multimídia

Permite a transferência de diversos tipos de recursos, incluindo páginas HTML, imagens, vídeos e arquivos de dados.



Importância do HTTP para a Web



Padronização

Criou uma linguagem comum que permitiu que diferentes sistemas e navegadores conversassem entre si sem barreiras.



Acessibilidade

Tornou a informação disponível globalmente, permitindo que qualquer pessoa com internet acesse conteúdos de qualquer lugar.



Interoperabilidade

Independente do hardware ou sistema operacional (Windows, Linux, macOS), o HTTP garante que a web funcione uniformemente.

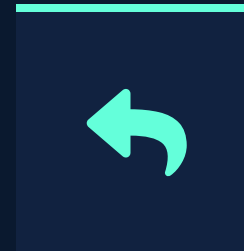
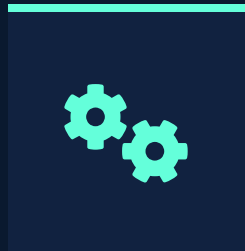
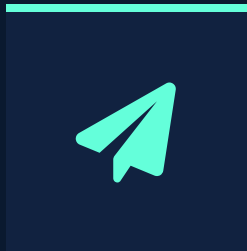


Crescimento

Foi o motor fundamental para a explosão de sites, e-commerces e aplicações web que definem a economia digital atual.

Modelo Cliente-Servidor

O ciclo fundamental de comunicação na Web



REQUISIÇÃO

O navegador (cliente) envia uma solicitação formal de um recurso ao servidor.

PROCESSAMENTO

O servidor recebe, interpreta o pedido e localiza o conteúdo nos seus arquivos.

RESPOSTA

O servidor envia o recurso solicitado (ou um erro) de volta para o cliente.

EXIBIÇÃO

O navegador renderiza o código recebido e exibe o conteúdo final ao usuário.

Request (Requisição) HTTP

Método (Method)

Define a ação a ser executada no recurso (ex: GET, POST).

Caminho (Path)

Endereço local do recurso no servidor (ex: /index.html).

Versão (Version)

Versão do protocolo HTTP utilizada (ex: HTTP/1.1).

Cabeçalhos (Headers)

Metadados com informações adicionais sobre a requisição.

EXEMPLO DE ESTRUTURA

```
GET /contato.html HTTP/1.1
Host: www.meusite.com.br
Accept-Language: pt-BR,pt;q=0.9
Connection: keep-alive
```

Response (Resposta) HTTP

EXEMPLO DE RESPOSTA

HTTP/1.1 200 OK

Date: Mon, 08 Jun 2026

Server: Apache/2.4.41

Content-Type: text/html

Content-Length: 155

<html>

 <body>Olá Mundo!</body>

</html>

Código de Status

Resultado da requisição (ex: 200, 404).

Cabeçalhos da Resposta

Metadados do servidor, tipo de conteúdo e cache.

Corpo (Body)

Conteúdo solicitado: HTML, JSON ou binários.

Protocolo

Versão do HTTP usada na resposta.

Linha do Tempo do HTTP

1996



HTTP/1.0

A primeira versão amplamente adotada, estabelecendo as bases da comunicação web.

1997



HTTP/1.1

Padronizou conexões persistentes (Keep-Alive) e suporte a múltiplos domínios.

2015



HTTP/2

Focado em performance através de multiplexação, compressão de cabeçalhos e formato binário.

2022



HTTP/3

Revolução com o protocolo QUIC baseado em UDP, eliminando latências e melhorando redes móveis.



HTTP/1.0 vs HTTP/1.1

HTTP/1.0

-  Uma nova conexão TCP para cada objeto solicitado.
-  Introduziu cabeçalhos e códigos de status básicos.
-  Alta latência e desperdício de recursos do servidor.

"O início da padronização, mas com limitações severas de escalabilidade."

HTTP/1.1

-  **Keep-Alive:** Conexões persistentes para múltiplos objetos.
-  **Virtual Hosts:** Vários sites em um único IP.
-  Controle de cache avançado e melhor aproveitamento da conexão.

"A versão que dominou a web por quase duas décadas, focada em eficiência."

HTTP/2 e HTTP/3: Velocidade e Eficiência

HTTP/2 (2015)



Multiplexação

Envia múltiplas requisições e respostas simultaneamente em uma única conexão TCP.



Compressão HPACK

Reduz o tamanho dos cabeçalhos, economizando largura de banda significativa.

Formato Binário

FOCO: PERFORMANCE EM CONEXÕES ESTÁVEIS

HTTP/3 (2022)



Protocolo QUIC

Substitui o TCP pelo UDP, eliminando o bloqueio de início de fila (Head-of-Line Blocking).



Resiliência Móvel

Transições suaves entre redes (ex: Wi-Fi para 4G) sem perder a conexão ativa.

Segurança Nativa

FOCO: REDUÇÃO DE LATÊNCIA E REDES INSTÁVEIS

Principais Métodos HTTP

GET



Consultar Informações

Utilizado exclusivamente para recuperar dados do servidor sem alterar seu estado. É o método mais comum na navegação web.

POST



Criar Recursos

Envia dados para o servidor para criar um novo recurso ou processar informações de formulários.

PUT



Atualizar Recursos

Substitui todas as representações atuais do recurso de destino pelos dados da requisição.

DELETE



Remover Recursos

Solicita a exclusão permanente do recurso especificado no servidor.

Estrutura das Mensagens HTTP

CABEÇALHOS COMUNS (HEADERS)

Host

Especifica o domínio do servidor (ex: www.google.com).

User-Agent

Identifica o navegador e sistema operacional do cliente.

Content-Type

Indica o formato dos dados enviados (ex: application/json).

Authorization

Contém credenciais para autenticação do usuário.

CORPO DA MENSAGEM (BODY)

Onde residem os dados reais da comunicação. Em requisições POST/PUT contém as informações enviadas pelo usuário; em respostas contém o recurso solicitado.

```
{  
  "usuario": "emilia_santos",  
  "disciplina": "Redes",  
  "status": "online"  
}
```

Códigos de Status HTTP

2xx

Sucesso

- ✓ **200** OK
- ✓ **201** Created
- ✓ **204** No Content

3xx

Redirecionamento

- ↪ **301** Moved Permanently
- ↪ **302** Found
- ↪ **304** Not Modified

4xx

Erro do Cliente

- × **400** Bad Request
- × **401** Unauthorized
- × **404** Not Found

5xx

Erro do Servidor

- ⚠ **500** Internal Server Error
- ⚠ **502** Bad Gateway
- ⚠ **503** Service Unavailable

Principais Servidores HTTP



APACHE

- ✓ O mais tradicional e amplamente utilizado.
- ✓ Altamente modular e fácil de configurar.
- ✓ Enorme comunidade e suporte técnico.

NGINX

- ✓ Focado em alto desempenho e concorrência.
- ✓ Baixíssimo consumo de memória RAM.
- ✓ Excelente como Proxy Reverso e Load Balancer.



LITESPEED

- ✓ Performance superior em processamento PHP.
- ✓ Otimização nativa para WordPress.
- ✓ Compatível com arquivos .htaccess do Apache.



CADDY

- ✓ HTTPS automático via Let's Encrypt por padrão.
- ✓ Configuração extremamente simples (Caddyfile).
- ✓ Escrito em Go, moderno e seguro.

Virtual Hosts



Hospedagem Múltipla

Permite que um único servidor web atenda a diversos nomes de domínio diferentes simultaneamente.



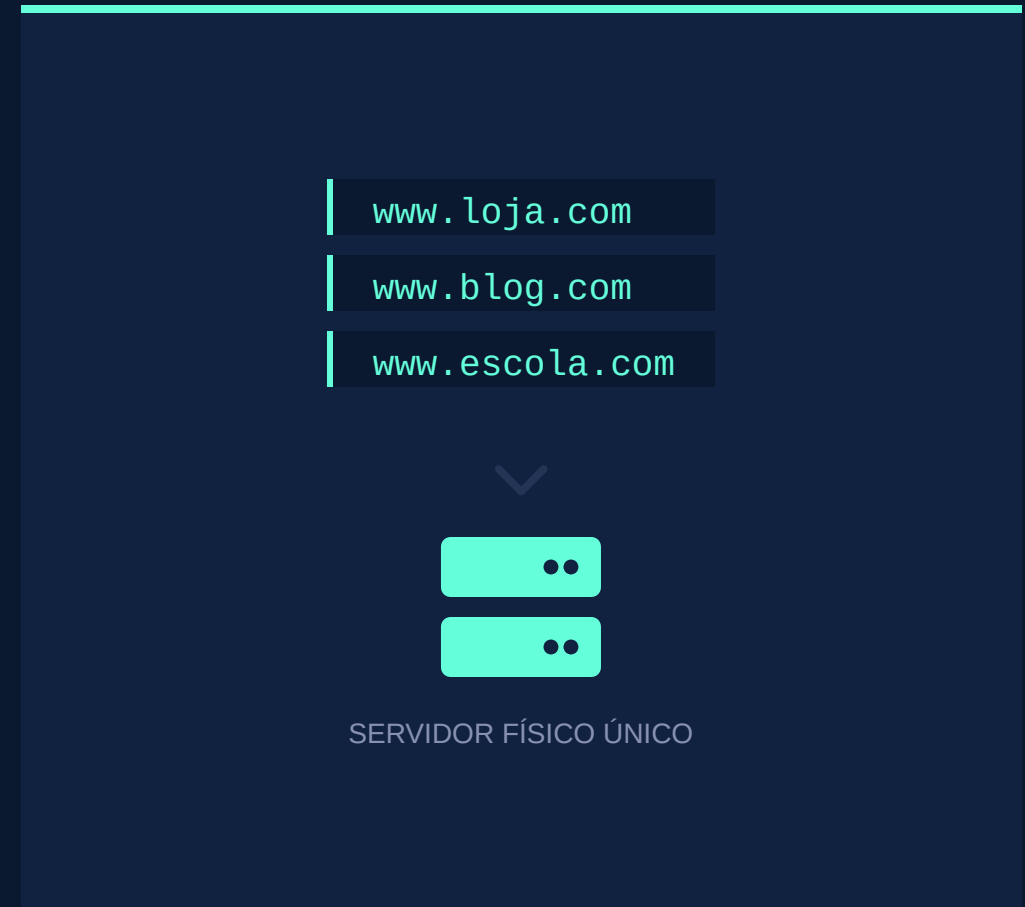
Economia de Recursos

Reduz drasticamente os custos de infraestrutura ao compartilhar CPU, memória e IP entre vários sites.



Escalabilidade

Facilita a gestão centralizada e o isolamento de configurações para cada aplicação hospedada.



HTTP vs HTTPS: Qual a Diferença?

Característica	HTTP	HTTPS
Segurança	 Inseguro (Texto Puro)	 Criptografado (SSL/TLS)
Porta Padrão	Porta 80	Porta 443
Certificado Digital	Não necessário	Obrigatório
Privacidade	Vulnerável a interceptação	Protegido contra espionagem
Integridade	Dados podem ser alterados	Garante que os dados são originais

Vantagens do HTTPS



Privacidade

Garante que a comunicação entre o cliente e o servidor seja privada, impedindo que terceiros interceptem dados sensíveis.



SEO e Ranking

Mecanismos de busca como o Google priorizam sites seguros em seus resultados, melhorando a visibilidade orgânica.



Autenticidade

Protege contra ataques de "Man-in-the-Middle", garantindo que o usuário está realmente conectado ao servidor legítimo.



Confiança do Usuário

O ícone do cadeado no navegador transmite segurança e profissionalismo, aumentando a credibilidade da marca.

Conclusão



Espinha Dorsal da Web

O HTTP permanece como o fundamento essencial para toda a comunicação e troca de dados na internet moderna.



Evolução para Performance

Do HTTP/1.0 ao HTTP/3, a evolução tem sido marcada pela busca por menor latência e maior eficiência.



Segurança como Padrão

O HTTPS tornou-se um requisito obrigatório, garantindo privacidade e integridade.

"O avanço do protocolo HTTP acompanha e impulsiona a evolução da própria Internet."

OBRIGADA!